egui-citizen — Book Plan

A plan for converting the current example-driven docs into a proper **mdBook**. The current `docs/getting-started.md` + three `examples/` subdirectories get you running but don't teach the design vocabulary or the non-obvious rules a user needs to build a real app.

This plan is calibrated against the crate's **actual public API as of 2026-04-21** (`src/lib.rs`, `src/citizen.rs`, `src/dispatcher.rs`, `src/message.rs`, `src/state.rs`) and against how CopperForge uses it.

Why a book, not more examples

The existing surface leaves three questions unanswered:

1. **Where does state live?** `CitizenState` holds reactive lifecycle fields (`active`, `clicked`, `visible`, etc.). Panels also have their own data (BOM cache, terminal buffer). Apps also have shared services. The line between these three is not drawn anywhere.
2. **Stored vs stateless panels?** CopperForge mixes both — `logger_panel`, `bom_panel`, `terminal_panel`, `shell_panel`, `gerber_view_3d_panel` are *stored* on the app struct, while `DrcPanel`, `ViewSettingsPanel`, `SettingsPanel`, `ProjectsPanel` are *constructed fresh every frame* in `tabs.rs`. The current docs don't tell you which to pick, or what breaks if you pick wrong.
3. **Reactivity pitfall.** `CitizenState::default()` / `::new()` is public and looks innocuous. But constructing a fresh `CitizenState` inline each frame creates *new `Dynamic<T>` handles disconnected from the dispatcher's handles* — reactivity silently breaks. The dispatcher's `register()` is the only way to get a state that's wired into the one-hot activation flow. This trap is currently undocumented.

A book with a narrative, chapter progression, and runnable examples is the right shape for teaching these.

mdBook structure

```
egui-citizen/
└── book/
    ├── book.toml
    ├── src/
    │   ├── SUMMARY.md
    │   ├── introduction.md
    │   ├── concepts/
    │   │   ├── problem.md
    │   │   ├── citizen.md
    │   │   ├── state.md
    │   │   ├── dispatcher.md
    │   │   └── messages.md
    │   ├── tutorial/
    │   │   ├── first-citizen.md
    │   │   ├── with-egui-dock.md
    │   │   └── two-panels.md
    │   ├── patterns/
    │   │   ├── stored-vs-stateless.md
```

```
│   │   ├── state-shape.md
│   │   ├── coordination.md
│   │   └── background-work.md
│   ├── pitfalls.md
│   ├── recipes/
│   │   ├── modal-dialog.md
│   │   ├── file-picker.md
│   │   └── long-running-task.md
│   └── reference.md
└── tests/          # mdbook-test or similar — runnable code samples
```

Deploys to GitHub Pages via `mdbook build` in CI.

Chapter-by-chapter plan

Introduction — who this book is for

Real-world framing: "You have an `egui_dock` app with three panels and they keep fighting over state. Here's why, and here's the library that fixes it."

Part 1: Concepts

Ch 1 — The problem

Why `egui_dock` alone produces per-frame races. Worked example: two panels both write to a shared `bool`, last render wins. Lead into the identity + dispatcher + message-queue answer. Source for this chapter: the "problem" paragraph in `lib.rs` lines 5–10.

Ch 2 — The `Citizen` trait

What an identity is (`CitizenId`) and why it's a stable string. The four hooks (`on_activate`, `on_deactivate`, `on_click`, plus `is_active` / `is_selected` readers). Minimum viable impl — five lines. Source: `citizen.rs`.

Ch 3 — `CitizenState` and reactivity

The six `Dynamic<T>` fields, what each one means, what "reactive" buys you over polling. **Crucial non-obvious rule:** cloning a `CitizenState` gives you a *handle to the same reactive storage* — constructing a fresh one does not. This chapter has to be crystal clear because it's the single biggest trap. Source: `state.rs`.

Ch 4 — The `Dispatcher`

`register()` returns a state clone that's wired into the dispatcher — hold onto it, pass it to the panel. `activate()` is an encoded set/reset — one live, rest off, atomically. `drain_messages()` is the backend boundary. Source: `dispatcher.rs`.

Ch 5 — `CitizenMessage` — the backend bridge

The six variants (`Activated`, `Deactivated`, `Clicked`, `Selected`, `Moved`, `VisibilityChanged`). How to forward to a thread via channels. When to extend with your own app-level message wrapper (CopperForge does `AppMessage::Citizen(CitizenMessage)` — worth showing as a pattern). Source: `message.rs`.

Part 2: Tutorial

Ch 6 — Your first citizen

Single panel, no dock. Goal: see `on_activate` fire and a `Dynamic<bool>` flip. Basis: adapt `examples/getting_started/`.

Ch 7 — Wiring into `egui_dock`

Implement `TabViewer`, route `on_tab_button` into `dispatcher.activate()`, drain messages after `DockArea::show()`. Basis: adapt `examples/citizen_dock/`. The key insight is *the dispatcher doesn't know about dock at all* — that boundary is user-code, by design.

Ch 8 — Two panels talking

One panel's activation drives another panel's content. Pure reactive: no messages, just `Dynamic<T>` reads. Teach by contrast with the "two panels writing the same state" problem from Ch 1 — now they can't race.

Part 3: Patterns

Ch 9 — Stored vs stateless panels

The chapter the docs are missing most. Two lawful ways to use a citizen:

- **Stored:** panel is a field on the app struct, constructed in `App::new()`, rendered via `self.panel.show(ui, ...)`. Use when the panel owns non-trivial local state (the log buffer, an image cache, a terminal history).
- **Stateless per-frame:** panel is constructed in the `TabKind` dispatch arm, `MyPanel::new(CitizenState::default()).show(ui, &mut app)`. Use only when the panel's entire state is read from `app` / `services` and the citizen's reactive fields aren't being subscribed-to from elsewhere.

The trap: the stateless form passes `CitizenState::default()`, which creates fresh disconnected `Dynamic<T>`s. If another panel / thread is trying to read `this_panel.state.active`, they read a different `Dynamic<bool>` than the one the dispatcher is updating — the reactive link is silently severed. For activation-only UIs this doesn't bite (the dispatcher's own state is still right), but for any panel that surfaces its citizen state as a reactive input elsewhere, **you must** use the stored form.

CopperForge's own `tabs.rs` is a good worked example of the split decision.

Ch 10 — What goes where

Three layers in a real app:

- **`CitizenState` (the library's):** lifecycle reactive bits only.
- **Panel struct fields:** panel-local non-reactive state (log buffer, UI filter text, modal open-state).

- **App-level shared state / `SharedServices`:** data that multiple panels read or mutate (layer store, project config, database handle).

Rule of thumb: if two panels need to see it, it's not panel-local; if it's a lifecycle fact (active? visible?), it's `CitizenState`; otherwise it's panel-local.

Ch 11 — Multi-citizen coordination

Beyond activation. How to pass richer messages: wrap `CitizenMessage` in your own `AppMessage` enum and enqueue your variants alongside citizen ones. Drain both in one loop. Source pattern: CopperForge's `messages::AppMessage::Citizen(CitizenMessage)`.

Ch 12 — Background work (start / cancel)

Long-running ops driven by `Activated` / `Deactivated`. Cancellation via dropping the worker or via an `AtomicBool` cancel flag. Basis: adapt `examples/citizen_fetch/` into a more realistic HTTP-with-cancel example. Needs a new example — the current fetch example likely doesn't cover cancellation cleanly.

Part 4: Pitfalls

Ch 13 — Common pitfalls

Each of these needs its own short section with a concrete broken snippet and the fix:

1. **Constructing `CitizenState` fresh per frame** severs reactivity. Use stored panels if the state is read elsewhere.
2. **Forgetting `drain_messages()`** — messages accumulate forever, memory grows.
3. **Calling `activate()` every frame unconditionally** — fires Activated/Deactivated messages every frame, floods the queue.
4. **Mixing panel-local state into `CitizenState`** — use panel struct fields instead.
5. **Expecting `visible` to track egui_dock's open/closed state automatically** — it doesn't. You must call `set()` on it yourself when a tab is closed/reopened, or route through `VisibilityChanged`.
6. **Two dispatchers in one app.** Don't. The one-hot invariant doesn't hold across dispatchers.

Part 5: Recipes

Ch 14 — Modal dialog as citizen state

Open/closed modeled as the citizen's `active`. Show by binding a modal's open flag to `state.active.get()`.

Ch 15 — File picker driven by activation

`Activated` → spawn file dialog, `Deactivated` → cancel.

Ch 16 — Long-running task with progress

Citizen activation starts a background thread, thread pushes progress messages into the dispatcher via `Dispatcher::send()` (which is already public in `dispatcher.rs` line 87), UI reads them out of

`drain_messages()`.

Reference

Auto-generated rustdoc link + a single-page cheat-sheet with the essential calls: `Dispatcher::new/register/activate/drain_messages/send`, `Citizen::on_*`, `CitizenState` field list, `CitizenMessage` variant list.

Code sample strategy

- Every code block in the book is runnable via `mdbook test` against a dummy `[dev-dependencies]` binary in `book/` so examples don't rot.
- The three existing `examples/` each expand into a tutorial chapter — we keep them as-is and reference them from the book.
- New examples the book needs that the repo doesn't have yet:
  - **`two_panels_reactive`** — Ch 8 material: panel A's activation switches panel B's content via reactive `Dynamic<T>` read.
  - **`stored_panel_log`** — Ch 9 material: a panel with a log buffer that survives across frames, demonstrating why "stored" is required.
  - **`fetch_with_cancel`** — Ch 12 material: start/cancel a background HTTP request via `Activated` / `Deactivated`.

What to do first

Don't write all 16 chapters before shipping. Order for first publish:

1. `book.toml` + SUMMARY.md scaffolding.
2. Introduction + Part 1 (Concepts) — 5 chapters. This alone is more guidance than the current docs provide.
3. Part 2 (Tutorial) — 3 chapters, each backed by one existing example.
4. Ch 9 (stored vs stateless) — the highest-value single chapter.
5. Ch 13 (pitfalls) — dump the gotchas from field use.
6. Everything else as time permits.

First publishable cut: chapters 1–9 + 13. ~10 chapters. Enough to let a new user build a real app without re-deriving everything from source.

Non-goals for v1

- Not teaching egui itself. Assume the reader can already put a slider on the screen.
- Not a design-patterns manifesto. The book is about using this crate well, not about GUI architecture generally.
- Not a migration guide from other frameworks. Maybe later.

Work sequencing

Do the book **after** any near-term API changes settle — otherwise chapters rot. Current API looks stable, but if `CitizenState` gains fields or `CitizenMessage` gains variants, update before freezing text.